

Code Explanation

Function Signature: The function accepts an integer `$decimal` and returns a string representing the binary form of the number.

Loop Through Bits: We start from the highest bit position (31 for a 32-bit integer) and iterate down to 0. This ensures we cover all bits of the number.

Bit Extraction: By creating a mask (`$mask`) for each bit position, we can check if the bit is set using the bitwise AND operator (`&`). If the result is not zero, the bit is set, and we append '1' to the binary string; otherwise, we append '0'.

Handling Edge Cases: This implementation assumes the input is a non-negative integer. For negative numbers, additional logic would be required to handle sign extension or two's complement representation, depending on the desired outcome.

Bitwise Shift Operators

Left Shift Operator (<<)

Purpose: The left shift operator moves the bits of the number to the left by a specified number of positions. Each shift to the left doubles the value of the number, effectively multiplying it by 2.

Usage: In the context of our function, the left shift operator is used to create a mask for each bit position during the conversion process. For example, `1 << $i` creates a mask where only the *i*th bit is set to 1, and all other bits are 0. This mask is then used to check if the corresponding bit in the decimal number is set.

Right Shift Operator (>>)

Purpose: The right shift operator moves the bits of the number to the right by a specified number of positions. Each shift to the right halves the value of the number, effectively dividing it by 2.

Usage: In our function, the right shift operator is used to traverse through the bits of the decimal number from the most significant bit (MSB) towards the least significant bit (LSB). After checking if a bit is set using the mask, the number is right-shifted to prepare for the next bit check.

Bitwise AND Operator (&)

Purpose: The bitwise AND operator compares each bit of the first operand to the corresponding bit of the second operand. If both bits are 1, the corresponding result bit is set to 1. Otherwise, the result bit is set to 0.

Usage: In our function, the bitwise AND operator is used in conjunction with the mask created by the left shift operator to determine if a particular bit in the decimal number is set. For example, `$decimal & $mask` checks if the *i*th bit of `$decimal` is set. If the result is not zero, it means the bit was set, indicating a '1' in the binary representation.

Combining Operations for Efficient Conversion

By combining these bitwise operations, the function efficiently traverses through all bits of the decimal number, starting from the MSB and ending at the LSB. It uses the left shift operator to generate masks for each bit position, the right shift operator to move through the bits, and the bitwise AND operator to check if a bit is set. This combination allows for a direct and efficient conversion of decimal to binary representation, leveraging the power of bitwise operations for bit manipulation.